

Activity 6: SwiftUI Controls and State

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K3, A3

Estimated time: 80 minutes

Objective

Build an accessible SwiftUI expense-entry shell whose controls reflect the domain contract.

Deliverable

SwiftUI entry flow, deterministic previews and validation states

Tools

SwiftUI, Observation, Xcode previews

Included files

- `starter/BudgetViewModel.swift`
- `starter/Category.swift`
- `starter/BudgetBridgeClient.swift`
- `starter/BudgetScreen.swift`
- `solution/BudgetScreen.swift`
- `solution/Category.swift`
- `solution/BudgetBridgeClient.swift`
- `solution/BudgetViewModel.swift`
- `solution/main.swift`
- `solution/preview-fixtures.txt`

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run `cd activities/06-swiftui-controls-state` and inspect both starter Swift files. Create a working copy with `cp -R starter workspace`.
2. Add `workspace/BudgetViewModel.swift` and `workspace/BudgetScreen.swift` to the app target that already contains the Activity 5 bridge.

3. In `BudgetViewModel`, keep draft text and category as editable state, but expose the accepted budget snapshot with `private(set)` so Views cannot bypass the bridge.
4. Implement a parser that converts the decimal amount string into integer cents. Reject empty, non-numeric, zero and negative input without clearing the learner's draft.
5. Implement `save()` to call `BudgetBridge`. Map the stable bridge error into a user-facing message and update the snapshot only when the domain result is accepted.
6. In `BudgetScreen`, add a `TextField` for formatted amount entry, a `Picker` for the closed category set and a `Button` for the explicit save consequence. Do not place budget policy in `body`.
7. Add `.accessibilityIdentifier("amount")`, `.accessibilityIdentifier("saveExpense")` and a semantic label for any icon-only control.
8. Create deterministic preview fixtures for empty, loaded and validation-error states. Add one preview with a large Dynamic Type environment.
9. Build the app and use the preview or Simulator to enter ``12.50``, select a category and save. Expect the bridge to receive ``1250`` cents and the accepted snapshot to refresh once.
10. Enter ``0``, ``-1`` and non-numeric text. Expect a clear validation message, no domain mutation and the original draft to remain available for correction.
11. Turn on VoiceOver or use Accessibility Inspector. Navigate the amount, category and save controls in logical order and confirm each announces purpose and current value.
12. Retain the two Swift files, four preview states, exact-cent observation, negative-path observation and accessibility note as the Activity evidence.

Verification

The view remains declarative, invalid input is recoverable, and previews cover four states without production data.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If previews fail, inject the fixture model and remove filesystem work from initialisation.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Retain preview fixtures; remove screenshots containing personal data.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.